

HW14 - GUI Programming with JavaFX

1. Main Functionalities

The application allows users to select a base date, choose a display format, and calculate a future or past date by adding or subtracting days, months, or years. It includes input validation, showing clear error messages for invalid data (e.g., text instead of numbers), and automatically disables the Apply button when the input is wrong.

2. Design Decisions (MVC Architecture)

I designed the application strictly following the MVC architectural pattern to separate logic from the user interface:

- **Model (DateModel):** Manages all the data and states, including the base date, format pattern, calculated result, and error messages.
- **View (DateView):** Builds the user interface programmatically using JavaFX layout panes (VBox, HBox) and UI controls. It contains no calculation logic.
- **Controller (DateController):** Acts as the bridge. It handles user events, validates input, updates the Model, and initializes bindings.

3. JavaFX Properties and Binding

In the DateModel, I used `ObjectProperty` and `StringProperty` instead of primitive Java variables. This allowed me to leverage JavaFX Binding effectively. A key feature is using `Bindings.createStringBinding` to link the current displayed date label to both the `baseDate` and `dateFormat` properties. The Apply button's `disable` property is also directly bound to the `isInputValid` property.

4. Problems and Improvements

Initially, I planned to update the result label manually using setter methods every time an event occurred. This approach made the code difficult to maintain. I improved this by binding the result label directly to the `resultDate` and `dateFormat` properties in the `initBindings()` method. Now, the interface is fully reactive and updates automatically when the model changes.